105TH ANNUAL MEETING
TRB 2026
Washington, D.C.
January 11-15, 2026

Repositioning in Bike Sharing Systems with Broken Bikes Considering On-site Repairs

Runqiu Hu[1], Jia Cui[1], W.Y. Szeto[1,2,3]
[1]Dept. of Civil Engineering, HKU [2]HKU Shenzhen Institute of Research and Innovation [3]GD-HK-Macau Joint Lab for Smart Cities

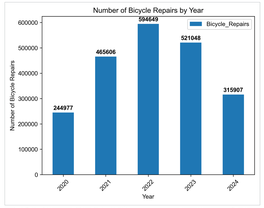
香 港 大 學
THE UNIVERSITY OF HONG KONG

1. Background, Motivation & Research Gap

The Challenge: Bike Sharing Systems (BSS) face critical operational issues:

Empty stations → Cannot rent

Full stations → Cannot return



Citi Bike: **21,637 bikes** repaired monthly

- Barcelona: 12% severe + 55% light damage
- New York: 2% need repairs *every day*
- Broken bikes occupy docks, reduce inventory

Research Gap: How are broken bikes currently handled?

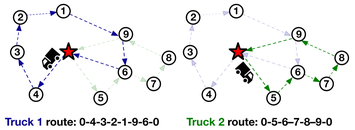
Off-site Repair (Depot only)
◄ Existing studies focus here

On-site Repair (Field)
► Real-world practice

Our Innovation: First to jointly optimize **Truck-based Repositioning** + **Repairer-based On-site Repairs**

2. Problem Description & Model

Problem Illustration:



(a) Spatial: Network with trucks & repairers

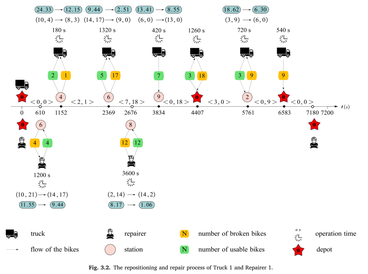


Fig. 3.2. The repositioning and repair process of Truck 1 and Repairer 1.

(b) Temporal: Repositioning & repair process

Mathematical Model (MILP):

Objective:
- User dissat. $F_i(\#usable, \#broken)$
- CO_2 emissions from trucks

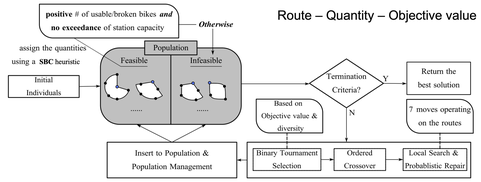
Key Decision Variables:
- Routings of trucks & repairers
- Usable bikes load/unload qty
- Broken bikes load/unload/repair qty

Key Constraints:
- Time budget T for all vehicles
- Truck capacity (usable + broken)
- Station inventory balance
- Flow conservation

Our Contributions:

Novel Problem	MILP Model	HGS-ADC+SBC	Insights
First to integrate on-site repairs	Time-indexed formulation	Scalable algorithm	Cost-effectiveness analysis

3. Our Approach: HGS-ADC with SBC



Key Features:

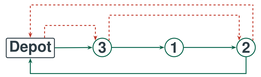
❶ Infeasible Population Kept
Explore beyond feasible boundaries

❷ Diversified Search
Avoid premature convergence

❸ Adaptive Control
Dynamic parameter adjustment

1. Route Decisions

Local Search: SWAP, 2-opt, Or-opt, Cross-exchange



Truck: Depot → 3 → 1 → 2 → Depot
Repairer: Depot → 3 → 2 → Depot

2. Quantity Assignment *The Hard Problem!*

	Station 1	Station 2	Station 3
Truck	u_1^+, u_1^-, b_1=?	u_2^+, u_2^-, b_2=?	u_3^+, u_3^-, b_3=?
Repairer	r_1=?	—	r_3=?

u^+: usable bikes loaded u^-: usable bikes unloaded b: broken bikes loaded r: broken bikes repaired

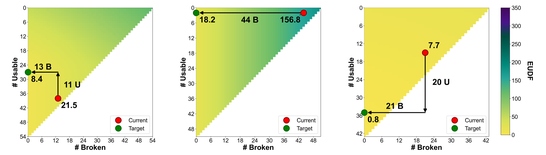
Each station has **different sensitivity** to inventory changes
✔ **Solution: SBC Allocation with BCRF**

BCRF: Benefit-Cost Ratio Function

Stn 1: Moderate
21.5−8.4=13.1 ⇒ **24** bikes
BCRF=$\frac{13.1}{24}$=**0.55**

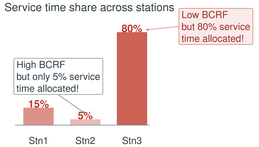
Stn 2: High
156.8−18.2=138.6 ⇒ **44** bikes
BCRF=$\frac{138.6}{44}$=**3.15**

Stn 3: Low
7.7−0.8−6.9 ⇒ **41** bikes
BCRF=$\frac{6.9}{41}$=**0.17**



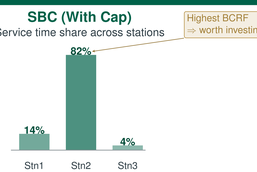
4. Key Innovation: Station-Budget-Constrained (SBC) Allocation

Greedy (No Cap)
Service time share across stations



✗ Early stations consume the remaining service time

SBC (With Cap)
Service time share across stations



✓ All stations are guaranteed nonzero service

Key idea: Use **BCRF** to pre-allocate **station-level service-time budgets** and enforce them as caps.

Step 1: Compute Benefit–Cost Ratio Function

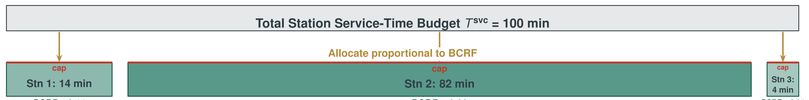
$$BCRF_s^t(p,b) = \frac{F_s(p,b) - F_s(q,0)}{t^{load} \cdot (|q-p| + b)}$$

$F_s(\cdot)$: dissat. at station s t^{load}: per-bike handling time
q: target usable bike inventory p: current usable bike inventory
b: broken bikes handled

Step 2: Allocate station budgets

$$B_s = T^{\text{SVC}} \cdot \frac{BCRF_s}{\sum_{j \in \mathcal{R}} BCRF_j}$$

T^{SVC}: total service time budget $\mathcal{R}$: visited stations
Higher BCRF ⇒ larger budget.



Step 3: Quantity assignment (what changes)

Actions must satisfy $t^{load} \cdot (|q-p| + b) \le$ cap.

Greedy: cap = **remaining time budget**
⇒ early stations can still monopolize time.

SBC: cap = **station budget B_s (+carryover)**
⇒ each station respects its budget cap.

SBC makes greedy quantity assignment **fair** by adding a **station-level time cap** driven by BCRF.

5. Numerical Studies

A) Solver Comparison (HGS vs Gurobi)

Small Instances
$|S| \le 15$, $|K|=|R|=1$
All solved to **optimality**.
CPU times (s): Gurobi vs HGS

	$\lvert S\rvert$ = 6	
inst.	Gurobi	HGS
1	13.41	6.29
2	13.85	5.75
3	66.97	6.22
4	2.29	3.63
5	22.29	4.84

	$\lvert S\rvert$ = 10	
inst.	Gurobi	HGS
1	928.50	3.87
2	1409.89	5.27
3	13768.72	3.02
4	2404.03	5.16
5	4172.75	4.17

	$\lvert S\rvert$ = 15	
inst.	Gurobi	HGS
1	2602.24	7.80
2	4209.67	4.89
3	6283.58	4.72
4	3374.38	5.32
5	2371.40	5.08

Large Instances (Scalability): $|S| = 60$–500, $K \in \{1,2\}$, $R \in \{1,2\}$.

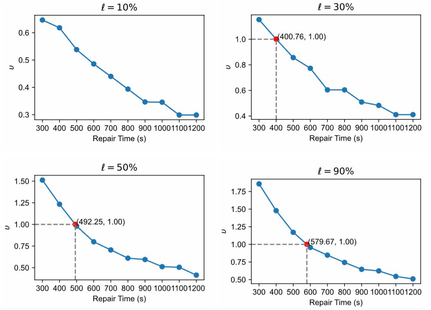
	K=1, R=1			K=1, R=2			K=2, R=1			K=2, R=2			CPU (s)	
$\lvert S\rvert$	O_H	O_G	Gap	O_H	O_G	Gap	O_H	O_G	Gap	O_H	O_G	Gap	Grb	HGS
							T = 2 **h**							
60	2581	2605	0.9	2494	2533	1.6	2401	3582	33.0	2325	3518	33.9	7200	6–11
90	4038	4511	10.5	3942	4159	5.2	3862	4511	14.4	3769	4489	16.0	7200	9–13
120	4303	5114	15.9	4238	5104	17.0	4168	5114	18.5	4097	5104	19.7	7200	9–16
200	7821	8657	9.7	7720	8657	10.8	7642	8657	11.7	7561	8657	12.7	7200	12–24
300	11169	11717	4.7	11058	11717	5.6	10989	11717	6.2	10888	11717	7.1	7200	17–34
400	14281	15069	5.2	14179	15061	5.9	14169	15069	6.0	14059	15061	6.7	7200	25–44
500	18962	19794	4.2	18793	19785	5.0	18752	19794†	5.3	18613	19785†	5.9	7200	27–51
							T = 3 **h**							
60	2433	3554	31.5	2365	3507	32.6	2265	3582	36.8	2214	3507	36.9	7200	10–14
90	3903	4504	13.3	3779	4481	15.7	3705	4504	17.7	3607	4481	19.5	7200	10–20
120	4203	5114	17.8	4107	5104	19.5	4024	5114	21.3	3952	5104	22.6	7200	9–18
200	7695	8657	11.1	7575	8657	12.5	7471	8657	13.7	7394	8657	14.6	7200	14–28
300	11024	11716	5.9	10901	11716	7.0	10830	11716	7.6	10693	11716	8.7	7200	18–44
400	14138	15069	6.2	14010	15061	7.0	13969	15069†	7.3	13852	15061†	8.0	7200	22–56
500	18818	19794	4.9	18596	19785†	6.0	18572	19794†	6.2	18407	19785†	7.0	7200	22–62

Setup & Notation
- CPU^{max} = 7200 s time limit, 20 runs per instance
- Repositioning time budget $T \in \{2, 3\}$ hours
- O_H: HGS objective (avg. over 20 runs)
- O_G: Gurobi upper bound
- Gap = $100 \times (O_G - O_H)/O_G$
- † Gurobi ran out of memory (64GB)

Key Findings
Small instances ($|S| \le 15$):
✓ HGS matches **optimal** solutions in all cases
✓ HGS: **3–8 seconds** vs Gurobi: **minutes–hours**
Large instances ($|S| = 60$–500):
✓ HGS improves Gurobi UB by up to **33.9%** within **6–62 seconds**
✓ Gurobi hits 2h time limit to find a worse solution or crashes (64GB)

B) When Are On-Site Repairers Cost-Effective?

Cost-effectiveness ν vs repair time (selected ℓ values):



Cost-Effectiveness Indicator

$$\nu = \frac{c^p \cdot \Delta F}{\varpi \cdot T/3600}$$

- ΔF: dissatisfaction reduction
- c^p: penalty cost per unit dissatisfaction
- ϖ: repairer hourly wage
- ν measures **benefit per wage**

Key Findings
✓ **Low** ℓ ($\le 20\%$)**:** repairers **never** pay off (curves always below $\nu = 1$)
✓ **Higher** $\ell \Rightarrow$ larger tolerance for slower repairs (**401–600 s**)
✓ **Practical rule:** Deploy repairers when $\ell \ge 30\%$ and repair time $\lesssim$ 7–10 min

6. Conclusions & Future Work

Conclusions:
✓ Joint truck + on-site repair routing reduces dissatisfaction and emissions.
✓ HGSADC–SBC scales to **500 stations** with **seconds-level** runtime.
✓ On-site repairers are cost-effective when the broken ratio is **> 30%** and the average repair time is **< 11 min**.

Future research:
★ Dynamic rebalancing with time-varying demand & stochastic breakdowns.
★ Predictive maintenance scheduling & rostering.
★ Dynamic repairer deployment should adapt to different damage levels.

Questions & discussions are welcome!

This research has been published in *Transportation Research Part E* 201, 104155.

Full Paper
(open access)
DOI:
10.1016/j.tre.2025.104155

Let's Connect!
LinkedIn
Runqiu Hu
The University of Hong Kong

Acknowledgments
This research was supported by:
- National Natural Science Foundation of China (71771194)
- Research Grants Council of Hong Kong (17206322)
- HKU University Research Committee (2202100693)

We thank the anonymous reviewers for their constructive comments.